

Plantilla de Informe - Taller de Contenerizacion y Despliegue con Docker

Fundamentos de Ingenieria de Software

- Docente: Luis Gabriel Moreno Sandoval, PhD
- Proyecto: DriveControl / AutoMinder Enterprise
- Fecha: ____ / ____ / 2026
- Curso/Grupo: _____

Integrantes

Nombre completo	GitHub	Rol	Firma (opcional)
Sebastian Ramirez Maldonado	@Sarm-m	Scrum Master	_____
Samuel Freile	@samuelfl680	Configuration Manager	_____
Sebastian Rodriguez Ramirez	@juserora	QA Lead	_____
Solon Losada	@solonlosada2006	DevOps Engineer	_____
Sebastian Vargas	@juanvargax	Product Owner y Sprint Planner	_____

Objetivo del laboratorio

Escribe en 5-8 lineas como el equipo uso Docker para contenerizar la solucion y automatizar su despliegue.

1) Uso del artefacto generado previamente

Evidencia esperada

- Confirmacion de build previo del proyecto (frontend y backend).
- Validacion de artefacto generado localmente.

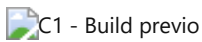
Comandos ejecutados

```
# Build frontend
cd apps/web
npm ci
npm run build

# Build backend (validacion minima)
cd ../../backend
npm ci
node --check server.js
```

Captura obligatoria C1 - Build previo

Inserta captura de terminal mostrando build exitoso del frontend y validacion backend.



C1 - Build previo

Analisis tecnico

- Que artefacto se genera:
- Donde se genera:
- Como se valida que es util para contenerizacion:

2) Dockerfile

Evidencia esperada

- Dockerfile correctamente estructurado.
- Imagen construida sin errores.
- Contenedor ejecutando aplicacion.

Archivo utilizado

- Ruta: Dockerfile

Explicacion por instruccion

- FROM:
- WORKDIR:
- COPY:
- EXPOSE:
- CMD/ENTRYPOINT:

Comandos ejecutados

```
# Construccion de imagen backend
docker build -t drivectrl-backend:local -f Dockerfile .

# Ejecucion de contenedor backend
docker run --name drivectrl-backend -p 5000:5000 \
  -e MONGO_URI="mongodb://host.docker.internal:27017/logistica_db" \
  -e JWT_SECRET="syntix_super_secret_key_dev" \
  drivectrl-backend:local
```

Captura obligatoria C2 - docker build

Inserta captura del comando docker build terminado en exitoso.



C2 - Docker build

Captura obligatoria C3 - contenedor ejecutando

Inserta captura de docker ps mostrando el contenedor activo.



C3 - Docker run / docker ps

Captura obligatoria C4 - endpoint de salud

Inserta captura de respuesta valida en navegador o Postman:

- <http://localhost:5000/api/health/db>

 C4 - Health backend

Analisis tecnico

- Decisiones de seguridad/tamano de imagen:
 - Variables de entorno usadas:
 - Problemas encontrados y solucion:
-

3) Docker Compose

Evidencia esperada

- Servicios definidos correctamente.
- Sistema completo levantado.
- Puertos, variables y dependencias funcionando.

Archivo utilizado

- Ruta: docker-compose.yml
- Servicios: mongodb, backend, frontend

Comandos ejecutados

```
# Levantar todos los servicios
docker compose up -d --build

# Ver estado de servicios
docker compose ps

# Ver logs (si hubo fallos)
docker compose logs backend --tail 100
```

Captura obligatoria C5 - compose up

Inserta captura de docker compose up -d --build finalizado.

 C5 - Compose up

Captura obligatoria C6 - compose ps

Inserta captura con los 3 servicios en estado running/healthy.

 C6 - Compose ps

Captura obligatoria C7 - frontend desplegado

Inserta captura del sistema abierto en:

- <http://localhost:3000>

 C7 - Frontend en contenedor

Analisis tecnico

- Como Compose orquesta el sistema:
 - Dependencias entre servicios:
 - Persistencia (volumen mongo_data):
-

4) Pipeline CI/CD con Docker (GitHub Actions)

Evidencia esperada

- Pipeline versionado y ejecutado.
- Build de imagenes automatizado.
- Publicacion preparada para DockerHub.

Archivo utilizado

- Ruta: .github/workflows/docker_ci_cd.yml

Flujo implementado

1. Validacion de docker-compose.
2. Build imagenes backend y frontend.
3. Publicacion condicional en DockerHub (si existen secretos).

Captura obligatoria C8 - workflow en Actions

Inserta captura de la ejecucion del workflow Docker CI/CD en verde.

 C8 - Actions workflow

Captura obligatoria C9 - detalle de jobs

Inserta captura donde se vea el job docker-validate completado.

 C9 - Jobs workflow

Analisis tecnico

- Como se integra Docker al pipeline:
 - Por que se usa publicacion condicional:
 - Mejoras futuras del pipeline:
-

5) Publicacion en DockerHub y validacion externa

Evidencia esperada

- Imagen publicada en DockerHub.
- Otro equipo descarga y ejecuta exitosamente.

Captura obligatoria C10 - repositorio DockerHub

Inserta captura de ambas imagenes publicadas.



C10 - DockerHub

Captura obligatoria C11 - validacion por otro equipo

Inserta captura o acta breve de ejecucion externa con fecha/equipo.



C11 - Validacion externa

Registro de validacion externa

- Equipo validador:
 - Fecha:
 - Resultado:
 - Observaciones:
-

6) Justificacion y analisis

Preguntas obligatorias

1. Que hace el Dockerfile y como esta estructurado.
2. Como Docker Compose permite ejecutar el sistema.
3. Como Docker se integra en el pipeline.
4. Que problemas surgieron.
5. Por que ocurrieron y como se solucionaron.

Redaccion (minimo 2 paginas)

Escribe aqui tu analisis tecnico completo:

[Espacio para desarrollo]

7) Mapeo directo con la rubrica

- Dockerfile (25%): evidencia C2, C3, C4 + explicacion tecnica.
 - Docker Compose (30%): evidencia C5, C6, C7 + analisis de dependencias.
 - Pipeline Docker CI/CD (15%): evidencia C8, C9 + diseno del flujo.
 - Funcionamiento del sistema (20%): evidencia C6, C7, C11.
 - Informe de resultados (10%): secciones 1-6 completas y coherentes.
-

8) Guia exacta de donde tomar capturas

1. C1: Terminal local, justo al terminar npm run build y node --check.
2. C2: Terminal local, salida completa de docker build del backend.

3. C3: Terminal local, resultado de docker ps con nombre del contenedor backend.
 4. C4: Navegador o Postman, endpoint de salud backend respondiendo ok.
 5. C5: Terminal local, docker compose up -d --build.
 6. C6: Terminal local, docker compose ps con estados running/healthy.
 7. C7: Navegador, aplicacion en <http://localhost:3000>.
 8. C8: GitHub Actions, lista de ejecuciones del workflow Docker CI/CD.
 9. C9: GitHub Actions, detalle interno del run con jobs completados.
 10. C10: DockerHub, repositorio con imagenes backend/frontend.
 11. C11: Evidencia de ejecucion por equipo externo (captura o constancia).
-

9) Checklist final antes de entregar

- ☐ Capturas C1 a C11 completas.
 - ☐ Analisis tecnico diligenciado.
 - ☐ Errores y soluciones explicados.
 - ☐ Evidencia de DockerHub incluida.
 - ☐ Evidencia de validacion externa incluida.
 - ☐ PDF exportado y revisado.
-

10) Exportar a PDF

En VS Code:

1. Abrir este archivo Markdown.
2. Abrir vista previa de Markdown.
3. Imprimir/Exportar a PDF.
4. Nombre sugerido: Informe_Taller_Docker_SYNTIX_2026.pdf